

Reporte Detallado: Módulo de Chatbot Web (Conector Web)

Elaborado por: Renzo

Sección Analizada: Panel de Web Chatbot (/dashboard/web-chatbot) y el propio Widget (/widget/[id])

Este informe documenta el funcionamiento del Chatbot Web, que permite a los clientes integrar un agente de Inteligencia Artificial directamente en sus páginas web (landing pages, tiendas) camuflado con la apariencia visual de WhatsApp.

1. Estructura y Funcionamiento en el Panel (La Pantalla)

El panel permite crear y configurar "Conexiones Web" o Widgets.

- **Vinculación de Agente:** Se selecciona qué "Cerebro" (Agente IA) atenderá los mensajes que entren por este widget.
- **Apariencia:** Permite elegir colores y un mensaje de bienvenida. La interfaz final imita el diseño de WhatsApp (incluyendo un aviso de "Cifrado de extremo a extremo" ficticio).
- **Distribución:** Proporciona un código de inserción (<script>) asíncrono para que el cliente lo pegue en su sitio web.

2. El Mapa Técnico: ¿Qué pasa por detrás?

La arquitectura de este módulo se divide en 3 grandes bloques:

A. El Inyector del Script (embed.js)

- **Ruta:** /api/widget/embed.js/route.ts
- Cuando la web del cliente carga este script, el código inyecta un botón flotante (burbuja) y un contenedor HTML invisible.
- Dentro de ese contenedor, carga un <iframe> que apunta de regreso a nuestra plataforma en la ruta /widget/[id].
- **Dato clave:** El script lee automáticamente todos los parámetros de la URL de la web del cliente (por ejemplo, ?email=juan@test.com) y se los transfiere al iframe. Esto permite pre-identificar al usuario si viene de una campaña de correo.

B. El Cliente Web (El iFrame)

- **Ruta:** /widget/[id]/page.tsx
- Es una aplicación React aislada que se ejecuta dentro del iFrame.
- Simula la interfaz de WhatsApp Web (estilos, doble check azul para lectura, "Escribiendo...").
- Maneja el historial de la conversación en la memoria temporal del navegador mientras el usuario chatea.

C. El Motor de Respuesta (Server Action widget.ts)

Cuando el usuario final envía un mensaje, el servidor ejecuta la función `getChatbotResponse`:

1. **Identificación / Captura de Lead:** Busca si existe un Lead con ese ID temporal, o busca en la base de datos si el usuario ingresó un Email o Teléfono (extraído de la URL). Si no existe, crea un nuevo Lead en la tabla lead.
2. **Armado de Contexto:** Reúne la memoria reciente (`ChatMemoryService`), el resumen de la conversación (`ChatSummaryService`) y busca en los PDFs subidos (`KnowledgeBaseService`).
3. **Ejecución del LLM:** Envía todo el paquete a OpenAI para obtener una respuesta.
4. **Registro:** Guarda el mensaje entrante y la respuesta de la IA en la tabla `chat_messages` para que el agente humano pueda verlo en el "Inbox".
5. **Extracción de Variables:** En segundo plano (sin bloquear el chat), le pide a la IA que lea el diálogo e intente extraer datos útiles (como nombres o correos) según lo que el cliente configuró en el Agente.

3. Fallos Críticos y Vulnerabilidades Detectadas

Durante el análisis del código, se detectaron riesgos de seguridad significativos y particularidades de implementación:

● FALLO CRÍTICO DE SEGURIDAD: API Abierta sin Rate Limits ni Whitelisting

- **El Problema:** La ruta que procesa los mensajes del chatbot (`getChatbotResponse` en `widget.ts`) es un endpoint completamente público. No valida de qué dominio (página web) proviene la solicitud (falta de CORS/Whitelisting de dominios) y no tiene límite de velocidad (Rate Limiting).
- **Impacto:** Cualquier atacante podría copiar el ID del widget del código fuente de la página del cliente y crear un script (botnet) que envíe miles de mensajes por segundo. Esto:
 1. Dispararía facturas masivas y sorpresivas en la cuenta de OpenAI del cliente (agotando su saldo).
 2. Llenaría la base de datos de leads basura (SPAM) y miles de conversaciones vacías en el Inbox.

💡 DETALLE TÉCNICO: Parche Manual del Modelo (gpt-4.1)

- **El Problema:** En el reporte de *Agentes de Texto*, detectamos que el sistema guardaba un modelo inexistente (gpt-4.1). Curiosamente, el desarrollador del módulo del Widget **sí fue consciente de este problema y aplicó un parche manual temporal** directamente en el archivo del widget (src/lib/actions/widget.ts Línea 150):
if (modelName === "gpt-4.1") modelName = "gpt-4o"; , esto fue por que han cambiado sus nombres o modelos dentro de open ai.
- **Impacto Positivo:** El Chatbot Web **sí funciona** y no se rompe a pesar del error de configuración en la creación del agente, ya que el código del widget lo reescribe al vuelo antes de llamar a OpenAI.

4. Resumen para el Cliente (Explicación Sencilla)

1. **Tu Widget funciona como WhatsApp, pero con Inteligencia Artificial:** Si lo pegas en tu web, los visitantes sentirán que chatean por WhatsApp. El bot lee tus PDFs (Base de Conocimiento) y les responde de inmediato, capturando sus datos automáticamente y guardándolos en tu CRM o Inbox.
2. **Es a prueba del "Error 4.1":** A diferencia de las pruebas internas que fallaban por el nombre del modelo de inteligencia artificial, este widget tiene una protección interna que corrige automáticamente ese error, por lo que tus clientes siempre recibirán respuesta.
3. ⚠️ **Advertencia de Seguridad (Riesgo de Spam):** Actualmente, el widget está "demasiado abierto". Necesitamos ponerle un escudo protector pronto, ya que si alguien malintencionado encuentra tu código, podría enviar mensajes falsos masivos, gastando el saldo de tu inteligencia artificial y llenando tu bandeja de entrada de "leads fantasmas".

